

DarkSage

Development Standards

Version 0.3.1 — Draft

Owner: TheSinnerMan

Classification: Internal

Wisdom Over Noise

Source repository: `LightSwornPVP/DarkSage`

Source baseline commit: `8b5cf5bd0341d5d6f70c67302c8bce2069d0929a`

Generation date: `2026-07-25T16:51:31-04:00`

Document ID: `DS-010`

Document Control

Document ID	DS-010
Title	Development Standards
Version	0.3.1
Status	Draft
Owner	TheSinnerMan
Classification	Internal
Repository	LightSwornPVP/DarkSage
Created	2026-07-24
Last Updated	2026-07-25

Status lifecycle: Draft → Under Review → Approved → Superseded/Deprecated.

Revision History

Version	Date	Author	Summary
0.3.1	2026-07-25	TheSinnerMan / Keeper	Independent-audit repair (H3): the Foundation Completion Delivery Standard section, previously misnumbered "## 24." (skipping §§16–23 after §15 References/Appendix A), renumbered to the correct next-available "## 16." No section content changed.
0.3.0	2026-07-25	TheSinnerMan / Keeper	Founder Vision Completion amendment and cross-volume traceability, including Discord notification boundaries where applicable.
0.1.0	2026-07-24	TheSinnerMan	First controlled draft, authored as part of the Batch 2 grouped pass (DS-010/DS-011/DS-012). Consolidates AGENTS.md, SECURITY_RULES.md's development-facing sections, docs/standards/*, and this Codex's own established authoring/audit practice into a single development-standards volume under the DS-DEV prefix.
0.2.0	2026-07-24	TheSinnerMan	Targeted repair for independent-audit findings DS-010-H1/H2. H1 : repaired DS-DEV-004's self-contradiction (claimed IDs are "never reused after removal" while also requiring renumbering on insertion) — controlled IDs are now permanently stable once assigned; document order may differ from numeric ID order; renumbering is permitted only pre-baseline or via an explicit, approved migration mapping. H2 : DS-DEV-006, DS-DEV-008, DS-DEV-009, DS-DEV-010, and DS-DEV-025 no longer cite .ai-workflow/** files as Governing Source — each now cites DS-010 itself (normative) or another public Codex/root source; local workflow files may implement a rule locally but never constitute its authority, consistent with DS-DEV-025's own rule. §13 updated to record this fix.
0.2.1	2026-07-24	TheSinnerMan	Consolidated cleanup pass: clarified DS-DEV-008's acceptance criteria, which previously read "no task requires more repair passes than the number of genuine Critical/High findings identified" — a criterion that was trivially satisfiable and did not mechanically capture the intended bundle-findings-into-one-pass discipline. Rewritten so the criterion is keyed to audit/verification cycles (one repair pass per cycle that identifies new findings) rather than raw finding count, and is checkable directly against each task's recorded blocker/finding history. The underlying blocker-only review philosophy and default workflow sequence are unchanged.

Table of Contents

Document Control	2
Revision History	3
Table of Contents	4
1. Purpose	6
2. Scope	6
3. Audience	6
4. Definitions	6
5. Coding and Design Principles	7
DS-DEV-001 — Earned Complexity and Code Quality	7
DS-DEV-002 — Repository Structure and Module Boundaries	7
DS-DEV-003 — Naming Conventions	8
DS-DEV-004 — Requirement and Controlled-ID Usage	8
6. Git and Review Discipline	9
DS-DEV-005 — Git Branch and Commit Conventions	9
DS-DEV-006 — Scoped Staging Rules	9
DS-DEV-007 — Code Review and Independent Audit Expectations	9
DS-DEV-008 — Minimum Practical Pass Philosophy	10
DS-DEV-009 — Critical/High Blocker Handling	10
DS-DEV-010 — Non-Blocking Suggestion Handling	11
7. Testing, Documentation, and Traceability	11
DS-DEV-011 — Testing Requirements	11
DS-DEV-012 — Documentation Requirements	11
DS-DEV-013 — Traceability Requirements	12
8. Dependencies, Environments, and Security Practice	12
DS-DEV-014 — Dependency Management	12
DS-DEV-015 — Reproducible Environments and Builds	12
DS-DEV-016 — Security Standards (Pointer)	13
DS-DEV-017 — Secrets Handling (Pointer)	13
DS-DEV-018 — Error Handling and Logging Practice	13
DS-DEV-019 — Deterministic Financial Calculation Coding Discipline	13
DS-DEV-020 — AI/Model-Use Boundaries for Development Tooling	14
9. Compatibility, Migration, and Observability	14

DS-DEV-021 — API Compatibility and Versioning Discipline	14
DS-DEV-022 — Database Migration and Data-Integrity Discipline	15
DS-DEV-023 — Performance and Observability Expectations	15
10. Repository Hygiene and Release Readiness	15
DS-DEV-024 — Tool-Neutral Public-Repository Requirements	15
DS-DEV-025 — Local-Only Workflow/Tool File Handling	16
DS-DEV-026 — Release-Readiness Expectations	16
11. Non-Goals	17
12. Dependencies	17
13. Risks and Constraints	17
14. Verification Approach	17
15. References	18
Appendix A — Open Questions	18
16. Foundation Completion Delivery Standard	18

1. Purpose

DS-010 is the authoritative statement of how DarkSage is built: coding and design principles, repository structure, naming and Git conventions, review and audit discipline, testing/documentation/traceability expectations, dependency and security practice, and release-readiness rules. It governs *how implementation work is carried out*; it does not restate the *what* — product requirements (DS-002), Sage behavior (DS-003), architecture (DS-004), data (DS-005), API contracts (DS-006), UI/UX (DS-007), security architecture (DS-008), or testing/QA (DS-009) remain authoritative for their own domains. Where a DS-010 requirement governs practice around one of those domains, it cites the domain volume rather than restating its content.

2. Scope

This document governs: coding and design principles; repository structure and module boundaries; naming conventions; requirement/controlled-ID usage; Git branch and commit conventions; scoped staging rules; code review and independent-audit expectations; the minimum-practical-pass philosophy; Critical/High blocker handling; non-blocking suggestion handling; testing, documentation, and traceability expectations at the process level; dependency management; reproducible environments/builds; security-practice pointers; secrets-handling practice; error-handling/logging practice; deterministic-financial-calculation coding discipline; AI/model-use boundaries for development tooling; API compatibility/versioning discipline; database migration/data-integrity discipline; performance/observability expectations; tool-neutral public-repository requirements; local-only workflow/tool file handling; and release-readiness expectations.

DS-010 does not govern: product-level feature commitments (DS-002), technical architecture decisions (DS-004, ADRs), database schema (DS-005), API contracts (DS-006), UI/UX behavior (DS-007), the security *architecture* itself (DS-008 — DS-010 states practice that implements it), or test *category* definitions (DS-009 — DS-010 states the process obligation that tests exist and are reviewed, not what they must cover).

3. Audience

All contributors and development tooling, present or future, working in the DarkSage repository; independent auditors; future Codex authors.

4. Definitions

See DS-001 §24 and DS-002 §4. Additional terms:

Term	Meaning
Earned Complexity	DS-001 §8.7/Constitution #11: a feature or abstraction is justified only by the meaningful user or engineering value it provides, not by anticipated future need

Term	Meaning
Scoped staging	Staging (<code>git add</code>) limited to the files a specific authorized task covers, verified against the task's authorized-file list before commit
Controlled document	Any DS-NNN Codex volume, ADR-NNN decision record, or other document under <code>docs/codex/</code> subject to this Codex's authoring/audit/approval lifecycle
Local-only workflow file	A file used for agent/session coordination (<code>.ai-workflow/**</code> , <code>LOCAL_CODER_RULES.md</code> , and local IDE/tool-configuration directories excluded via <code>.gitignore</code>) that is never committed and never authoritative project governance

5. Coding and Design Principles

DS-DEV-001 — Earned Complexity and Code Quality

Release Classification: Committed / MVP | **Governing Source:** DS-001 §8.7 (Constitution #11, Committed); `AGENTS.md` "Code Quality"

Description: Code shall prioritize clarity, maintainability, type safety, testability, explicit behavior, small modules, and clear interfaces over cleverness. A feature, abstraction, or dependency is added only when it provides meaningful, demonstrable value (Earned Complexity) — not for anticipated future need or novelty. Avoid clever code when simple code is safer.

Acceptance Criteria:

- A code review that identifies unjustified complexity (an abstraction with a single call site, a speculative extension point with no current consumer) is a valid basis for requesting simplification before merge.
- No Committed/MVP module trades correctness or clarity for premature optimization without a documented, measured justification.

Testing: Code-review checklist item; static-analysis/type-checking gate in CI where configured.

DS-DEV-002 — Repository Structure and Module Boundaries

Release Classification: Committed / MVP | **Governing Source:** `AGENTS.md` "Specialist Ownership," "File Ownership Boundaries"; `ARCHITECTURE.md` §5 (module tree)

Description: Repository structure follows `ARCHITECTURE.md`'s module tree (`apps/desktop/`, `apps/mobile/`, `backend/app/*`, `shared/`, `ai/*`, `tests/`) and `AGENTS.md`'s specialist-ownership boundaries (Frontend/Mobile/Backend/Quant/Trading/AI/QA). Cross-boundary changes require coordination rather than unreviewed edits into another specialist's owned area; unrelated edits outside a task's scope are avoided.

Acceptance Criteria:

- A change touching a module outside its author's typical ownership area is called out explicitly in the change's summary.

- No trading-logic module exists under a client application directory (apps/desktop/, apps/mobile/), per DS-ARC-001/DS-ARC-002/DS-ARC-003's already-Committed boundary.

Testing: Architectural boundary test (shared with DS-ARC-001's own test); code-ownership/import-boundary static analysis.

DS-DEV-003 — Naming Conventions

Release Classification: Committed / MVP | **Governing Source:** docs/standards/NAMING_AND_ID_STANDARD.md

Description: Controlled documents use DS-NNN; requirements use DS-<DOMAIN>-NNN; architecture decisions use ADR-NNN; change proposals use DSCP-NNN; design reviews use DR-NNN. Controlled filenames use stable kebab-case names (e.g., DS-001-Executive-Vision.md) and never embed mutable version/status text.

Acceptance Criteria: Matches NAMING_AND_ID_STANDARD.md exactly; a new controlled document or requirement family is rejected in review if it does not conform.

Testing: Filename/ID-format lint check across docs/codex/.

DS-DEV-004 — Requirement and Controlled-ID Usage

Release Classification: Committed / MVP | **Governing Source:** DS-002 §5.1 (Committed); DS-012 (ADR ID governance)

Description: A requirement ID's NNN component is a zero-padded integer within its domain family, assigned the next available (highest-plus-one) number in that family at the time of creation. Once assigned, a controlled ID is **permanently stable**: it is never renumbered merely to preserve visual or document-reading sequence, and a retired/removed requirement's ID is marked Withdrawn, never reused for different content (DS-002 §5.1). Document order — the order requirements appear reading top-to-bottom — may differ from numeric ID order; a requirement inserted logically between two existing ones receives the next available ID for its family, not a renumbered slot between them. Cross-references, docs/traceability/TRACEABILITY_MATRIX.csv entries, tests, and any external reference to a controlled ID remain valid for that ID's lifetime, since the ID itself never changes. Renumbering an already-assigned ID is permitted only (a) before that ID has entered a controlled/public baseline — i.e., before its containing document has been committed — or (b) through an explicitly approved migration recorded with an old-ID-to-new-ID mapping, reserved for a rare, explicitly governance-approved exception; it is never performed silently or merely for cosmetic sequencing.

Acceptance Criteria:

- No controlled document in this Codex contains a duplicate requirement ID (verified by grep-based unique-ID check on every authoring/repair pass).
- No committed controlled ID is renumbered without an explicit, approved migration record and mapping; a document's numeric ID order differing from its reading order is expected behavior, not a defect requiring correction.
- A new requirement inserted logically ahead of existing ones in reading order receives the next available ID for its family, never a renumbered predecessor's slot.

Testing: Unique-ID grep per document, performed on every authoring/repair pass; ID-stability check confirming no previously committed ID has been reassigned to different content across revisions.

6. Git and Review Discipline

DS-DEV-005 — Git Branch and Commit Conventions

Release Classification: Committed / MVP | **Governing Source:** AGENTS.md "Branching," "Commit Guidelines"

Description: Major feature changes avoid direct commits to main, using focused feature branches (e.g., feature/backend-foundation, fix/order-idempotency). Commits are clear and focused (e.g., feat: add candle data model, fix: prevent duplicate paper orders), avoiding vague messages.

Acceptance Criteria: Matches AGENTS.md's branching/commit examples; a commit message lacking a clear type/summary is flagged in review.

Testing: Commit-message convention lint (where configured); branch-naming review checklist item.

DS-DEV-006 — Scoped Staging Rules

Release Classification: Committed / MVP | **Governing Source:** DS-010 (this document, normative); AGENTS.md "File Ownership Boundaries"

Description: Staging (`git add`) is scoped to exactly the files a task's authorized-file list covers. `.ai-workflow/**`, `LOCAL_CODER_RULES.md`, and local IDE/tool-configuration directories (DS-DEV-025) are never staged. Before committing, staged name/status is checked against the authorized scope; `git diff --cached --check` is run to catch whitespace errors. This obligation is normative on DS-010's own authority; a local session-coordination file that happens to also describe this practice is an implementation convenience, never the source of the rule.

Acceptance Criteria:

- No commit stages a file outside its task's authorized scope.
- No commit stages a local-only workflow file.

Testing: Pre-commit `git status --short/git diff --cached --name-status` review against the authorized-file list, performed on every commit.

DS-DEV-007 — Code Review and Independent Audit Expectations

Release Classification: Committed / MVP | **Governing Source:** AGENTS.md "Critical Code Review," "AI-Assisted Code Review"; SECURITY_RULES.md "Independent Review"; DS-SCA-021 (Committed)

Description: Major changes are reviewed before merging. Security-critical code (DS-008 §4 definition) requires independent review — no single agent, human or AI, both implements and solely approves it, per DS-SCA-021. Generated code is reviewed like human-written code; it is never assumed correct merely because a model produced it. This Codex's own Author/Repair-Agent-vs.-Independent-Auditor separation (`.ai-workflow/AGENT_PROTOCOL.md`) is the documentation-process instantiation of this same principle and extends identically to product code.

Acceptance Criteria: Matches DS-SCA-021's acceptance criteria exactly, extended to all major (not only security-critical) changes at the "reviewed before merge" level.

Testing: Code-review process audit (shared with DS-SCA-021's own test).

DS-DEV-008 — Minimum Practical Pass Philosophy

Release Classification: Committed / MVP | **Governing Source:** DS-010 (this document, normative); DS-QA-017, DS-QA-019 (both Committed)

Description: The default workflow is Author/Implement → Self-verify → One independent Critical/High audit → Repair blockers only if necessary → Final verification → Approve/commit. Additional passes are used only when genuinely necessary — as few passes as practical, as many as necessary. Unnecessary cosmetic review cycles are not created for their own sake. This sequence is normative on DS-010's own authority; any local session-coordination file that also records a version of it (e.g., an operational delegated-authority workflow) implements this practice locally but is never its source.

Acceptance Criteria:

- All Critical/High findings identified by a single independent audit or verification cycle are addressed together in one corresponding repair pass; findings from the same audit cycle are not serialized into separate repair passes without a documented reason (e.g., an explicit Keeper/Owner scope-narrowing decision).
- A new repair pass begins only when a subsequent audit, focused verification, or Keeper/Owner-directed review identifies a Critical/High finding not already covered by the prior repair pass — never merely to re-check work already verified clean.
- The number of repair passes recorded for a task therefore equals the number of distinct audit/verification cycles that identified at least one new Critical/High finding, not the raw count of individual findings — mechanically checkable against the task's recorded blocker/finding history (DS-DEV-009): each recorded repair-pass entry must trace to the specific audit cycle and finding set it resolved.
- A repair pass addresses only the findings it was scoped to; it does not silently expand scope.

Testing: Process audit against a task's recorded blocker/finding history, confirming each repair-pass entry traces to exactly one audit/verification cycle and its full finding set (shared with DS-DEV-009's own test).

DS-DEV-009 — Critical/High Blocker Handling

Release Classification: Committed / MVP | **Governing Source:** DS-QA-019 (Committed, defect severity scale); DS-010 (this document, normative)

Description: A Critical or High finding (per DS-QA-019's severity scale) blocks the affected capability's or requirement's release/commit until resolved. A blocker is recorded — with its severity, resolution, approver, and commit hash once resolved — in the project's controlled issue-tracking record for code, or the equivalent controlled document record for documentation. A local session-coordination file, where used, may implement this recording locally for a given work session, but the obligation to record and resolve Critical/High findings before release is DS-010's own normative rule, independent of any particular local tool.

Acceptance Criteria: Matches DS-QA-019's severity-scale acceptance criteria; no Critical/High finding is silently dropped without a recorded resolution or an explicit, approved disposition.

Testing: Blocker-log completeness audit (shared with DS-QA-019's own test).

DS-DEV-010 — Non-Blocking Suggestion Handling

Release Classification: Committed / MVP | **Governing Source:** DS-010 (this document, normative); DS-QA-019 (Committed)

Description: Useful suggestions that do not rise to Critical/High severity (DS-QA-019) remain welcome and are recorded for a future consolidated cleanup pass rather than triggering an immediate, separate repair cycle. Non-blocking findings never prevent a batch from reaching independent audit or, once audited clean at Critical/High, from being approved. This is DS-010's own normative rule; a local session-coordination file, where used, may record specific non-blocking items for a given work session, but does not itself define the obligation.

Acceptance Criteria:

- A non-blocking suggestion is recorded (e.g., in the relevant volume's Appendix A / Risks and Constraints, or an equivalent controlled or project-tracking record) rather than discarded.
- No batch is held from audit or commit solely because non-blocking suggestions remain outstanding.

Testing: Review-log audit confirming non-blocking items are recorded, not silently dropped or silently escalated into a blocking pass.

7. Testing, Documentation, and Traceability

DS-DEV-011 — Testing Requirements

Release Classification: Committed / MVP | **Governing Source:** DS-009 (Committed, authoritative for test categories); AGENTS.md "Testing"

Description: Every meaningful feature includes tests where practical, per DS-QA-001's baseline obligation. Critical areas (indicators, backtesting, Risk Engine, permissions, order execution, broker adapters, authentication, portfolio accounting, strategy statistics) require strong coverage. DS-010 states the process obligation that tests exist and are reviewed; DS-009 is authoritative for which test categories apply and what they must verify.

Acceptance Criteria: Matches DS-QA-001's acceptance criteria; a Committed/MVP feature merging without an associated test (per its Testing field) is rejected in review.

Testing: Requirements-completeness audit (shared with DS-QA-001's own test).

DS-DEV-012 — Documentation Requirements

Release Classification: Committed / MVP | **Governing Source:** AGENTS.md "Documentation"; DS-001 §11 (Constitution #12, Codex-Driven Engineering)

Description: Major features enter the DarkSage Codex before implementation (DS-001 Constitution #12) and update relevant documentation as part of the same change. Documentation describes current behavior, not imaginary or aspirational behavior — a Planned or Future/Exploratory capability is labeled as such, never presented as already built.

Acceptance Criteria:

- No major feature merges without a corresponding Codex entry (new requirement, updated requirement, or ADR) existing first or in the same change.
- No documentation describes a capability's behavior as current when its Release Classification is Planned or Future/Exploratory.

Testing: Documentation-currency review checklist item.

DS-DEV-013 — Traceability Requirements

Release Classification: Committed / MVP | **Governing Source:** DS-QA-018 (Committed); DS-002 §5.5

Description: This requirement restates the process obligation DS-QA-018 already defines at the testing-architecture level: every Committed/MVP requirement across the Codex maps to at least one recorded test in docs/traceability/TRACEABILITY_MATRIX.csv, extending the five-stage model (Requirement → Design/ADR → Source → Test → Release/Change).

Acceptance Criteria: Matches DS-QA-018's acceptance criteria exactly.

Testing: Traceability-completeness audit (shared with DS-QA-018's own test).

8. Dependencies, Environments, and Security Practice

DS-DEV-014 — Dependency Management

Release Classification: Committed / MVP | **Governing Source:** AGENTS.md "Dependencies," "External Services"; DS-SCA-018 (Committed)

Description: Before adding a dependency, contributors evaluate necessity, maintenance status, security, cost, standard-library alternatives, and deployment complexity, avoiding dependency bloat. Paid APIs, cloud databases, hosted caching services, paid charting, cloud GPUs, and subscription services are not added automatically without explicit approval; user-configured cloud AI providers (DS-ARC-014) are the sole exception, since the user supplies their own key and DarkSage never bundles or defaults to one.

Acceptance Criteria: Matches DS-SCA-018's acceptance criteria exactly, extended to the explicit AGENTS.md "External Services" exception for user-configured cloud AI.

Testing: Dependency-review checklist item; dependency-scanning test (shared with DS-SCA-018's own test).

DS-DEV-015 — Reproducible Environments and Builds

Release Classification: Committed / MVP | **Governing Source:** ROADMAP.md Phase 0 exit criteria ("Development environment reproducible"); PROJECT_SPEC.md §2.1 (Cheap-First Architecture)

Description: The development environment is reproducible from committed configuration (dependency manifests, environment templates) without undocumented manual setup steps, consistent with ROADMAP.md Phase 0's exit criteria and the Cheap-First Architecture principle (prefer deterministic local code and open-source tooling; Stage 1 development requires no paid hosting service, per DS-ARC-018).

Acceptance Criteria:

- A clean checkout can reach a runnable local development state using only committed configuration and documented setup steps.
- No Committed/MVP development workflow requires an undocumented external service or paid tool.

Testing: Clean-checkout setup verification (shared with ROADMAP.md Phase 0's exit criterion and DS-ARC-018's Stage-1 zero-cost verification).

DS-DEV-016 — Security Standards (Pointer)

Release Classification: Committed / MVP | **Governing Source:** DS-008 (Committed, authoritative)

Description: All code follows DS-008's security architecture in full. DS-010 does not restate DS-008's controls; it states the process obligation that they are followed and reviewed as part of normal development, per DS-SCA-021's independent-review requirement for security-critical code.

Acceptance Criteria: No merge introduces a security-relevant change without a corresponding DS-008 control being satisfied or, where a gap is found, a recorded finding.

Testing: Security test suite execution (shared with DS-QA-013's own test).

DS-DEV-017 — Secrets Handling (Pointer)

Release Classification: Committed / MVP | **Governing Source:** DS-SCA-002, DS-SCA-003 (both Committed, authoritative)

Description: No secret, credential, or API key is ever committed, logged, or exposed in frontend/client source, per DS-SCA-002/003. Development uses .env files excluded by .gitignore; production uses OS-appropriate secure storage. DS-010 restates this as a coding-practice rule rather than redefining DS-008's architecture.

Acceptance Criteria: Matches DS-SCA-002/003's acceptance criteria exactly.

Testing: Secret-scanning test (shared with DS-SCA-002/003's own test).

DS-DEV-018 — Error Handling and Logging Practice

Release Classification: Committed / MVP | **Governing Source:** AGENTS.md "Error Handling"; DS-OPS-001, DS-OPS-003 (both Committed); DS-SCA-001 (Committed)

Description: Critical errors are never silently ignored. Trading-relevant code fails closed (DS-SCA-001) rather than guessing. User-facing errors follow DS-OPS-003's plain-language standard; material application events are logged per DS-OPS-001, excluding secrets (DS-SCA-002).

Acceptance Criteria: Matches DS-OPS-001/003 and DS-SCA-001's acceptance criteria exactly.

Testing: Error-message content audit; fail-closed adversarial test (shared with the respective requirements' own tests).

DS-DEV-019 — Deterministic Financial Calculation Coding Discipline

Release Classification: Committed / MVP | **Governing Source:** DS-PRD-004 (Committed); DS-QA-004, DS-QA-005 (both Committed); ADR-003

Description: Material financial calculation code is deterministic — identical inputs produce identical outputs across runs and versions, verified by a fixture-based regression suite (DS-QA-005) before merge. No material financial calculation ships as generative-model output, per ADR-003/DS-PRD-004. The same indicator, risk, strategy, and performance logic has one authoritative implementation (AGENTS.md "No Duplicate Business Logic") — no feature reimplements calculation math independently.

Acceptance Criteria: Matches DS-PRD-004/DS-QA-005's acceptance criteria exactly, plus the no-duplicate-implementation rule.

Testing: Deterministic-output regression suite (shared with DS-QA-005's own test); cross-consumer determinism test (shared with DS-ARC-007's own test).

DS-DEV-020 — AI/Model-Use Boundaries for Development Tooling

Release Classification: Committed / MVP | **Governing Source:** AGENTS.md "AI-Assisted Code Review"; DS-PRD-001, DS-PRD-011 (both Committed); DS-003 Core Rule

Description: Development tooling that produces code or documentation through a model (code-generation or documentation-authoring agents, including this Codex's own Author/Repair Agent) is reviewed like human-written work, never assumed correct by virtue of the producing tool (DS-DEV-007). Output produced this way is subject to the same independent-review discipline as any other contribution. This requirement governs *development tooling's* use of a model; Sage's own product-facing AI boundaries are DS-003's exclusive concern and are not restated here.

Acceptance Criteria:

- No change produced by development tooling (code or Codex documentation) is merged/approved based solely on the producing tool's own self-assessment.
- This Codex's own Author/Repair-Agent-vs.-Independent-Auditor separation is maintained for every controlled-document change, mirroring this requirement for documentation as a concrete, already-operating instance.

Testing: Independent-review process audit (shared with DS-DEV-007's own test).

9. Compatibility, Migration, and Observability

DS-DEV-021 — API Compatibility and Versioning Discipline

Release Classification: Committed / MVP | **Governing Source:** DS-API-COR-003 (Committed); AGENTS.md "Shared Contracts"

Description: A breaking change to a Committed/MVP API contract requires a new version, not an in-place silent change, per DS-API-COR-003. Changes to shared models or API contracts update all affected clients, backend services, tests, and migration notes in the same change (AGENTS.md

"Shared Contracts").

Acceptance Criteria: Matches DS-API-COR-003's acceptance criteria; a PR changing a shared contract without updating all affected consumers is rejected in review.

Testing: Cross-client/backend schema-consistency test (shared with DS-ARC-005's own test); contract-conformance test (shared with DS-QA-003's own test).

DS-DEV-022 — Database Migration and Data-Integrity Discipline

Release Classification: Committed / MVP | **Governing Source:** AGENTS.md "Database Changes"; DS-ARC-016 (Committed)

Description: Once migrations are introduced, schema changes use them rather than ad hoc manual edits. Destructive schema changes require impact review and a documented migration/backup path. A future database-engine migration (e.g., SQLite → PostgreSQL, DS-ARC-016's Phase 12 direction) requires the same reviewed migration/backup path, not an in-place undocumented switch.

Acceptance Criteria: Matches DS-ARC-016's acceptance criteria exactly, plus the migration-tooling requirement once migrations exist.

Testing: Migration-review checklist item; local functional test against the current database engine (shared with DS-ARC-016's own test).

DS-DEV-023 — Performance and Observability Expectations

Release Classification: Committed / MVP | **Governing Source:** DS-NFR-001 (Committed); DS-SCA-020, DS-SCA-025, DS-SCA-027 (all Committed)

Description: Committed/MVP performance budgets (DS-NFR-001's startup budget, and DS-NFR-002's interaction-latency budget once Planned) are regression-tracked, not assessed subjectively. Material application events, security-sensitive actions, and Risk Engine/Sage-affecting determinations are logged and auditable per DS-SCA-020/025/027, giving contributors and operators the observability needed to diagnose behavior without exposing secrets.

Acceptance Criteria: Matches DS-NFR-001 and DS-SCA-020/025/027's acceptance criteria exactly.

Testing: Automated startup-time regression benchmark (shared with DS-NFR-001's own test); audit-log integrity/observability tests (shared with DS-SCA-025/027's own tests).

10. Repository Hygiene and Release Readiness

DS-DEV-024 — Tool-Neutral Public-Repository Requirements

Release Classification: Committed / MVP | **Governing Source:** AGENTS.md Source-of-Truth priority-order note; scripts/verify-foundation.sh's prohibited-phrase scan

Description: Public repository content — source code, code comments, commit messages, and committed documentation — does not name any specific development-tool product or vendor, and does not imply the codebase or its correctness depends on a particular tool. Local, tool-specific, or machine-local development-tool configuration (tracked or untracked) is a convenience summary only; it never becomes authoritative project governance merely because a development tool reads it automatically, and it never overrides AGENTS.md's fixed document-priority order. **"The DarkSage Codex" and its `DS-NNN`/`ADR-NNN` terminology are legitimate project terminology, not a prohibited tool reference** — this document, and scripts/verify-foundation.sh's prohibited-phrase scanner, treat that distinction as settled: the Codex is DarkSage's own governance system, unrelated to any specific development-tool product.

Acceptance Criteria:

- scripts/verify-foundation.sh's prohibited-phrase scan passes on every commit touching public repository content.
- No commit message, code comment, or committed document names a specific development-tool product as an authority over project governance or as project branding.
- References to "the DarkSage Codex," DS-NNN, or ADR-NNN are never flagged or treated as violations of this requirement.

Testing: scripts/verify-foundation.sh prohibited-phrase scan (existing repository tooling, run on every authoring/repair pass in this Codex's history).

DS-DEV-025 — Local-Only Workflow/Tool File Handling

Release Classification: Committed / MVP | **Governing Source:** DS-010 (this document, normative); AGENTS.md Source-of-Truth priority-order note

Description: .ai-workflow/**, LOCAL_CODER_RULES.md, and the local IDE/tool-configuration directory are local-only coordination files: they are never committed, never staged (DS-DEV-006), and never authoritative project governance, per this document's own authority — a local file's existence or its own claims never elevate it to a governing source (AGENTS.md's priority-order note). The local IDE settings file and its root-level local instructions file are git-ignored, per scripts/verify-foundation.sh's existing checks.

Acceptance Criteria: Matches scripts/verify-foundation.sh's existing git-ignore checks exactly; no local-only workflow file is ever staged (DS-DEV-006) or cited as a Governing Source by any Committed/MVP requirement in this Codex.

Testing: scripts/verify-foundation.sh git-ignore checks (existing repository tooling).

DS-DEV-026 — Release-Readiness Expectations

Release Classification: Committed / MVP | **Governing Source:** DS-QA-017 (Committed, authoritative for release gates)

Description: This requirement restates the process obligation DS-QA-017 already defines: a phase's ROADMAP.md exit criteria and DS-009's applicable test categories must pass before that phase is recorded complete. DS-010 does not redefine the gate content; it states that release-readiness review is a required development-process step, not an optional formality.

Acceptance Criteria: Matches DS-QA-017's acceptance criteria exactly.

Testing: Release-gate checklist execution (shared with DS-QA-017's own test).

11. Non-Goals

DS-010 does not: redefine any DS-002 through DS-009 requirement's substantive content (it governs practice around them); select a specific CI/CD platform, linter, or formatter (an implementation detail, though its outputs are referenced where they already exist, e.g., `scripts/verify-foundation.sh`); define the ADR governance system (DS-012's concern) or the roadmap's phase content (DS-011's concern); or introduce a new security control beyond what DS-008 already defines.

12. Dependencies

- DS-001, DS-002, DS-004, DS-006, DS-008, DS-009
- `AGENTS.md`, `SECURITY_RULES.md`, `PROJECT_SPEC.md`, `ROADMAP.md`
- `docs/standards/NAMING_AND_ID_STANDARD.md`, `DOCUMENTATION_STANDARD.md`, `STYLE_GUIDE.md`, `WRITING_GUIDE.md`, `BRAND_GUIDE.md`
- `.ai-workflow/AGENT_PROTOCOL.md`, `KEEPER_AUTHORITY.md`
- `scripts/verify-foundation.sh`

13. Risks and Constraints

- **Consolidation, not duplication:** most DS-DEV-NNN requirements restate an already-Committed upstream obligation (from `AGENTS.md`, `SECURITY_RULES.md`, or DS-006/008/009) as a development-process rule rather than inventing new substantive content, consistent with the instruction not to duplicate every technical requirement from earlier volumes.
- **Public authority, not local-file authority (repaired in the DS-010-H2 pass):** DS-DEV-006, DS-DEV-008, DS-DEV-009, and DS-DEV-010 previously cited `.ai-workflow/**` files as their Governing Source, contradicting DS-DEV-025's own rule that local workflow files are never authoritative. Each now cites DS-010 itself (or another public Codex/root source) as its Governing Source; a local session-coordination file may still *implement* a rule locally, but it is never the rule's *source*. This closes the self-contradiction without weakening any of the underlying practice (scoped staging, minimum-practical-pass discipline, blocker handling, non-blocking-suggestion capture all remain unchanged in substance).
- **Classification discipline:** every requirement here is Committed/MVP because development process applies from the first line of code, regardless of which product phase a given feature belongs to; no requirement promotes a Planned product capability (DS-011 governs phase classification) to Committed by virtue of being a development rule.

14. Verification Approach

Each DS-DEV-NNN requirement states its own Testing. Document-level verification (unique-ID check, cross-reference consistency against DS-001/DS-002/DS-004/DS-006/DS-008/DS-009, no Committed requirement depending on a Planned-only capability) recorded in `.ai-workflow/HANDOFF.md`.

15. References

- AGENTS.md, SECURITY_RULES.md, PROJECT_SPEC.md, ROADMAP.md
- docs/standards/*
- docs/codex/Volume-08-Security/DS-008-Security-Architecture.md, Volume-09-Testing/DS-009-Testing-and-QA.md
- .ai-workflow/AGENT_PROTOCOL.md, KEEPER_AUTHORITY.md
- scripts/verify-foundation.sh

Appendix A — Open Questions

1. **CI/CD tooling/platform selection** — explicitly out of scope for this document (§11); a future DS-010 revision may adopt one once chosen.
2. **Linter/formatter selection** — not fixed here; DS-DEV-001's clarity/maintainability principles apply regardless of tooling choice.
3. **Governance-confirmation carryover** — the standing BLOCKERS.md items (ROADMAP.md phase boundaries as Codex release-scope authority; phase-mapping precision) apply identically to this document's Release Classification scheme and are not re-litigated here.

16. Foundation Completion Delivery Standard

Governance shall protect product value without creating unnecessary ceremony. For a bounded change, the normal path is: author or repair; self-verify; one independent Critical/High blocker audit; one repair pass if needed; final verification; approval; commit. Additional cycles require a real blocker, ambiguity, or material risk.

New product domains must trace from DS-001 through requirements, architecture, data, API, UX, security, testing, and roadmap before being represented as implementation-ready. Release artifacts must be regenerated from approved Markdown and accompanied by a manifest recording source commit/version, generated files, checksums, validation results, and known limitations.